

Course: Object Oriented Programming

Assignment 2

Inheritance

Task 1:

Create a base class called Animal with the following:

- An attribute name.
- A method speak() that prints "This is a generic animal." Create a derived class called Dog

that:

- Overrides the speak() method to print "Woof Woof" **only if the name attribute is set to "Buddy."**
- If the name is not "Buddy," the Dog class should call the base class speak() method.

1. Instantiate an object of the Animal class and call its speak() method.
2. Instantiate an object of the Dog class with the name attribute set to "Buddy" and call its speak() method.
3. Instantiate another object of the Dog class with the name attribute set to "Max" and call its speak() method.

Sample output:

```
This is a generic animal.  
Woof Woof.  
This is a generic animal.
```

Task 2:

Create a base class Vehicle with:

- An attribute speed.
- A method displaySpeed() that prints the speed.

Create a derived class Car that:

- Adds an attribute brand.
 - Adds a method displayBrand() that prints the brand.
 - Ensures that the brand attribute can only be set if the speed is greater than 0. If the speed is 0, the brand should default to "Unknown."
1. Instantiate an object of the Car class with speed = 0 and call both displaySpeed() and displayBrand().

2. Instantiate another object of the Car class with speed = 100 and brand = "Toyota", then call both methods.

Sample Output:

```
Speed: 0
Brand: Unknown
Speed: 100
Brand: Toyota
```

Task 3:

Create a base class Shape with:

- An attribute name.
- A method display() that prints the shape's name.

Create two derived classes:

1. Circle:

- Adds a method calculateArea() that calculates and prints the area of the circle (use πr^2 with $r = 5$).
- The method should only calculate the area if the name is "Circle." Otherwise, it should print "Invalid shape."

2. Rectangle:

- Adds a method calculateArea() that calculates and prints the area of the rectangle (use length * width with length = 6 and width = 4).
- The method should only calculate the area if the name is "Rectangle." Otherwise, it should print "Invalid shape."

1. Instantiate an object of the Circle class with name = "Circle" and call both display() and calculateArea().
2. Instantiate an object of the Rectangle class with name = "Rectangle" and call both display() and calculateArea().
3. Instantiate an object of the Circle class with name = "Triangle" and call both display() and calculateArea().

Sample Output:

```
Shape: Circle
Area: 78.5
Shape: Rectangle
Area: 24
Shape: Triangle
Invalid shape.
```

Task4:

Create a base class Employee with:

- Attributes name and salary.
- A method showDetails() that prints the name and salary.

Create a subclass Manager that:

- Overrides showDetails() to include an additional attribute department.

Create another subclass SeniorManager that:

- Overrides showDetails() to include an additional attribute performanceRating.
 - Ensures that the performanceRating is only displayed if the salary is greater than 100,000. Otherwise, it should print "Performance rating not available."
1. Instantiate an object of the Employee class and call showDetails().
 2. Instantiate an object of the Manager class and call showDetails().
 3. Instantiate an object of the SeniorManager class with a salary of 90,000 and call showDetails().
 4. Instantiate another object of the SeniorManager class with a salary of 150,000 and call showDetails().

Sample Output:

```
Name: John, Salary: 50000
Name: Alice, Salary: 120000, Department: HR
Name: Bob, Salary: 90000, Department: Finance, Performance Rating: Not available
Name: Carol, Salary: 150000, Department: IT, Performance Rating: Excellent
```

Task 5:

Create a base class Appliance with the following:

1. A method turnOn() that prints "Appliance is turned on."

A method turnOff() that prints "Appliance is turned off."

3. A property isOperational that returns True by default.

Design two subclasses, Fan and Light, with the specifications below:

Class 1: Fan Attributes:

- isSummer (boolean): Indicates if the current season is summer.
- isBroken (boolean): Indicates if the fan is broken.

Method Overrides:

1. turnOn()

- If the fan is operational **and** it is summer: Print "Fan is turned on." ○ If the fan is operational **but** it is not summer: Print "Fan cannot be turned on in winter." ○ If the fan is **not** operational: Print "Fan is broken and cannot be turned on."

2. **turnOff()**: Print "Fan is turned off."

3. **isOperational property**: Returns True only if isBroken is False.

Class 2: Light

Attributes:

- isDaytime (boolean): Indicates if it is daytime.
- powerLevel (integer): Represents the light's brightness level (0 to 100).

Method Overrides:

1. **turnOn()**

- If the light is operational **and** it is not daytime: Print "Light is turned on at [powerLevel]% brightness." ○ If the light is operational **but** it is daytime: Print "Light cannot be turned on during the day." ○ If the light is **not** operational: Print "Light is broken and cannot be turned on."

2. **turnOff()**: Print "Light is turned off."

3. **isOperational property**: Returns True only if powerLevel > 0.

Instantiate the following objects and call their methods to produce the **exact output** below:

1. A Fan with isSummer=False and isBroken=False. Call turnOn() and turnOff().
2. A Light with isDaytime=False and powerLevel=75. Call turnOn() and turnOff().
3. A Fan with isSummer=True and isBroken=True. Call turnOn() and turnOff().
4. A Light with isDaytime=True and powerLevel=0. Call turnOn() and turnOff().

Sample Output:

```
Fan cannot be turned on in winter.
Fan is turned off.
Light is turned on at 75% brightness.
Light is turned off.
Fan is broken and cannot be turned on.
Fan is turned off.
Light is broken and cannot be turned on.
Light is turned off.
```